离散数学：归纳与递归 (Induction and Recursion) 复习讲义

整理自：DS-chapter5-QYT-2026 PPT

目录

1 全章导引 **3**

2 数学归纳法、强归纳法与良序性 **3**

2.1 适用对象与核心思想 . . . 3
2.2 基本步骤与推理形式 . . . 3
2.3 归纳法为什么有效 . . . 4
2.4 例 1：前 n 个正整数之和 . . . 4
2.5 例 2：指数增长不等式 . . . 5
2.6 例 3：有限集合的子集数 . . . 5
2.7 强归纳法与良序性 (Strong Induction and Well-Ordering) . . . 5
2.8 强归纳法 . . . 5
2.9 例 4：整数可分解为素数乘积 . . . 6
2.10 例 5：4 分和 5 分邮票 . . . 6
2.11 良序性 . . . 6
2.12 例 6：用良序性证明带余除法 . . . 7
2.13 例 7：循环赛中的三元环 . . . 7
2.14 无限递降法 (Infinite Descent) . . . 7

3 递归定义与结构归纳 (Recursive Definitions and Structural Induction) **8**

3.1 递归定义函数 . . . 8
3.2 例 8：简单递归函数与阶乘 . . . 8
3.3 Fibonacci 数列 . . . 9
3.4 Euclidean Algorithm 与 Lame 定理 . . . 9
3.5 递归定义集合与结构 . . . 10
3.6 合式公式的递归定义 . . . 11
3.7 递归定义树 . . . 12
3.8 结构归纳法 . . . 12
3.9 例 9：递归集合 S 是正的 3 的倍数集合 . . . 12

3.10 例 10：合式公式中的括号数量 13
3.11 例 11：字符串长度的连接公式 13
3.12 例 12：满二叉树的顶点数上界 13
3.13 广义归纳法 (Generalized Induction) 14
3.14 递归算法 (Recursive Algorithms) 15
3.15 定义与设计方法 15
3.16 例 13：递归计算幂 15
3.17 例 14：递归计算模幂 15
3.18 例 15：递归计算最大公约数 16
3.19 例 16：递归二分查找 16
3.20 递归与迭代 16
3.21 归并排序 (Merge Sort) 17

4 程序正确性 (Program Correctness) 19
4.1 基本概念 19
4.2 循环不变式 19
4.3 递归程序正确性 20

5 全章总结 20

1 全章导引

本章围绕“如何证明无限多个对象上的命题”与“如何用自身定义自身”两条主线展开。归纳法提供证明工具，递归定义提供对象与算法的构造方式，二者本质上互相配合。

Chapter Summary

1. **数学归纳法 (Mathematical Induction)**；
2. **强归纳法 (Strong Induction)**；
3. **良序性 (Well-Ordering)**；
4. **递归定义 (Recursive Definitions)**；
5. **结构归纳法 (Structural Induction)**；
6. **递归算法 (Recursive Algorithms)**；
7. **程序正确性 (Program Correctness)**。

2 数学归纳法、强归纳法与良序性

2.1 适用对象与核心思想

数学归纳法用于证明形如

$$\forall n\, P(n)$$

的命题，其中论域通常是正整数集合。它非常适合处理求和公式、不等式、计数结论、递归序列性质等问题。

- **重要提醒**：归纳法通常用于证明已经猜出或通过其他方式发现的结论，而不是直接发现公式或定理的工具。
- **无限梯子类比**：若能到达第一阶梯，并且“能到达第 k 阶”总能推出“能到达第 $k+1$ 阶”，则可以到达任意有限高度的阶梯。

2.2 基本步骤与推理形式

要证明 $P(n)$ 对所有正整数 n 为真，通常包含两步：

1. **基础步 (Basis Step)**：证明 $P(1)$ 为真。
2. **归纳步 (Inductive Step)**：证明对任意正整数 n，若 $P(n)$ 为真，则 $P(n+1)$ 为真。

作为推理规则，可写为

$$\big[P(1) \wedge \forall n\big(P(n) \to P(n+1)\big)\big] \to \forall n P(n).$$

归纳证明模板

归纳步中只是假设某个任意的 $P(k)$ 为真，这个假设称为**归纳假设 (inductive hypothesis)**。并没有假设所有 $P(n)$ 已经为真，因此归纳证明不是循环论证。
若命题从 $n = n_0$ 开始成立，则基础步改为证明 $P(n_0)$，归纳步证明 $P(k) \to P(k+1)$ 对所有 $k \geq n_0$ 成立。

2.3 归纳法为什么有效

数学归纳法的有效性来自正整数的**良序性**：每个非空的正整数子集都有最小元素。
证明思路如下：

- 已知 $P(1)$ 为真，并且 $P(k) \to P(k+1)$ 对所有正整数 k 成立。
- 反设存在某个正整数使 $P(n)$ 为假。令

$$S = \{n \in \mathbb{Z}^+ \mid P(n) \text{ 为假}\}.$$

- 若 S 非空，由良序性，S 有最小元素 m。
- 因为 $P(1)$ 为真，所以 $m \neq 1$，从而 $m-1$ 是正整数。
- 由 m 的最小性，$P(m-1)$ 为真；又由归纳步，$P(m)$ 必为真，矛盾。

因此 $P(n)$ 对所有正整数 n 成立。

2.4 例 1：前 n 个正整数之和

证明

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

证明：令 $P(n)$ 表示上述等式成立。

- **基础步**：$P(1)$ 为真，因为

$$\sum_{i=1}^{1} i = 1 = \frac{1(1+1)}{2}.$$

- **归纳步**：假设 $P(k)$ 为真，即

$$\sum_{i=1}^{k} i = \frac{k(k+1)}{2}.$$

则

$$\sum_{i=1}^{k+1} i = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}.$$

因此 $P(k+1)$ 为真。

2.5 例 2：指数增长不等式

证明对所有正整数 n，有

$$n < 2^n.$$

证明：令 $P(n)$ 为命题 $n < 2^n$。

- **基础步**：$1 < 2^1 = 2$。
- **归纳步**：假设 $k < 2^k$。则

$$k + 1 < 2^k + 1 \leq 2^k + 2^k = 2^{k+1}.$$

故 $n < 2^n$ 对所有正整数 n 成立。

2.6 例 3：有限集合的子集数

证明：若有限集合 S 有 n 个元素，其中 n 是非负整数，则 S 有 2^n 个子集。
证明：令 $P(n)$ 表示“每个含 n 个元素的集合都有 2^n 个子集”。

- **基础步**：$n = 0$ 时，空集只有一个子集，即它本身；而 $2^0 = 1$。
- **归纳步**：假设任意 k 元集合有 2^k 个子集。设 T 是一个 $k+1$ 元集合，取 $a \in T$，令 $S = T - \{a\}$，则 $|S| = k$。
- 对 S 的每个子集 X，在 T 中对应两个子集：

$$X, \qquad X \cup \{a\}.$$

因此 T 的子集数为 $2 \cdot 2^k = 2^{k+1}$。

课件练习

P350: 44, 61。

2.7 强归纳法与良序性 (Strong Induction and Well-Ordering)

2.8 强归纳法

强归纳法在归纳步中允许使用从起点到当前所有命题作为假设：

1. **基础步**：证明 $P(1)$ 为真。
2. **归纳步**：证明

$$P(1) \wedge P(2) \wedge \cdots \wedge P(n) \to P(n+1)$$

对每个正整数 n 成立。

强归纳法与普通数学归纳法等价。区别只在于证明时可使用的归纳假设更强，因此特别适合处理“当前对象依赖多个更小对象”的问题。

2.9 例 4：整数可分解为素数乘积

证明：每个大于 1 的整数都可以写成素数的乘积。

证明：令 $P(n)$ 表示 “整数 n 可以写成素数的乘积”。

- **基础步**：$P(2)$ 为真，因为 2 本身是素数。
- **归纳步**：假设 $P(j)$ 对所有 $2 \le j \le k$ 成立。考虑 $k+1$：
  - 若 $k+1$ 是素数，则结论直接成立。
  - 若 $k+1$ 是合数，则存在整数 a, b，使得

$$k+1 = ab, \qquad 2 \le a \le b < k+1.$$

由归纳假设，a 与 b 都可写成素数乘积，因此 $k+1$ 也可写成素数乘积。

这证明了存在性。素因子分解的**唯一性**还需要额外证明。

2.10 例 5：4 分和 5 分邮票

证明：每个不少于 12 分的邮资都可以只用 4 分和 5 分邮票组成。

证明：令 $P(n)$ 表示 “n 分邮资可由 4 分和 5 分邮票组成”。

- **基础步**：

$$12 = 4+4+4, \quad 13 = 4+4+5, \quad 14 = 4+5+5, \quad 15 = 5+5+5.$$

- **归纳步**：假设 $P(j)$ 对所有 $12 \le j \le k$ 成立，其中 $k \ge 15$。要证明 $P(k+1)$。

  因为 $k-3 \ge 12$，由归纳假设，$k-3$ 分可组成。再加一张 4 分邮票，即可组成

$$(k-3)+4 = k+1$$

分。

2.11 良序性

良序性 (well-ordering property)：每个非空的非负整数集合都有最小元素。

这个性质可以推广到其他集合：若集合的每个非空子集都有最小元素，则称该集合在相应次序下是**良序的**。

- $\mathbb{N}$ 在通常的 $\le$ 下是良序的。
- 有限字母表上的有限字符串集合，在合适的字典序或长度优先字典序下也可形成良序结构。
- 良序性是归纳法、强归纳法、无限递降法的共同基础。

2.12 例 6：用良序性证明带余除法

带余除法定理：若 a 是整数，d 是正整数，则存在唯一整数 q, r，使得

$$a = dq + r, \qquad 0 \le r < d.$$

存在性证明：令

$$S = \{a - dq \mid q \in \mathbb{Z},\ a - dq \ge 0\}.$$

当 q 取足够小的整数时，$a-dq$ 足够大，因此 S 非空。由良序性，S 有最小元素，记为 $r = a-dq_0$。

若 $r \ge d$，则

$$a - d(q_0 + 1) = a - dq_0 - d = r - d \ge 0,$$

并且 $r - d < r$，这与 r 的最小性矛盾。因此 $0 \le r < d$。

唯一性补充：若

$$a = dq + r = dq' + r', \qquad 0 \le r, r' < d,$$

则 $d(q - q') = r' - r$。右端绝对值小于 d，又是 d 的倍数，只能为 0，故 $r = r'$ 且 $q = q'$。

2.13 例 7：循环赛中的三元环

在循环赛中，每两个选手恰好比赛一次，并且一胜一负。若存在长度 $m \ge 3$ 的环：

$$p_1 \text{ 胜} p_2, \quad p_2 \text{ 胜} p_3, \quad \ldots, \quad p_m \text{ 胜} p_1,$$

则必存在长度为 3 的环。

证明思路：

- 反设不存在三元环。
- 所有存在环的长度构成一个非空正整数集合。由良序性，取最短环长度 k，且 $k > 3$。
- 考察最短环中的 p_1, p_2, p_3。若 p_3 胜 p_1，则 p_1, p_2, p_3 构成三元环，矛盾。
- 因此只能是 p_1 胜 p_3。于是

  $$p_1, p_3, p_4, \ldots, p_k, p_1$$

  构成更短的环，仍然矛盾。

2.14 无限递降法 (Infinite Descent)

无限递降法常用于证明某个命题 $P(k)$ 对所有正整数 k 都为假。基本逻辑是：

- 若存在某个正整数 k 使 $P(k)$ 为真，则由良序性，存在最小的这种正整数 s。
- 再构造一个更小的正整数 $s' < s$，使 $P(s')$ 也为真。
- 这与 s 的最小性矛盾，因此不存在这样的 k。

例：用无限递降法证明 $\sqrt{2}$ 是无理数

反设 $\sqrt{2}$ 是有理数。则可取正整数 M, N，使

$$\sqrt{2} = \frac{M}{N},$$

并且在所有这种表示中选取分母 N 最小者。

因为 $1 < \sqrt{2} < 2$，所以

$$0 < M - N < N.$$

又由 $M^2 = 2N^2$ 可验证

$$\frac{2N - M}{M - N} = \frac{M}{N} = \sqrt{2}.$$

于是得到一个分母为 $M - N$ 的新表示，且 $0 < M - N < N$，这与 N 的最小性矛盾。因此 $\sqrt{2}$ 是无理数。

课件练习

P363: 12, 25, 31, 37。

3 递归定义与结构归纳 (Recursive Definitions and Structural Induction)

3.1 递归定义函数

对定义域为非负整数的函数，递归定义通常包含：

1. **基础步**：指定函数在 0 或前若干个初始整数处的值。
2. **递归步**：用较小整数处的函数值定义较大整数处的函数值。

递归定义的良定义性

若基础值已经给定，且每一步只依赖更小位置上已经确定的函数值，则可用数学归纳法证明该函数在每个非负整数处都是良定义的。

3.2 例 8：简单递归函数与阶乘

若函数 f 递归定义为

$$f(0) = 3, \qquad f(n+1) = 2f(n) + 3,$$

则

$$f(1) = 9, \qquad f(2) = 21, \qquad f(3) = 45.$$

阶乘函数 $F(n) = n!$ 的递归定义为

$$F(0) = 1, \qquad F(n+1) = (n+1)F(n).$$

3.3 Fibonacci 数列

Fibonacci 数列定义为

$$f_0 = 0, \qquad f_1 = 1, \qquad f_n = f_{n-1} + f_{n-2} \quad (n = 2, 3, 4, \ldots).$$

例：Fibonacci 数列的下界

令

$$\alpha = \frac{1+\sqrt{5}}{2}.$$

证明当 $n \geq 3$ 时，

$$f_n > \alpha^{n-2}.$$

证明：使用强归纳法。

- **基础步**：

$$f_3 = 2 > \alpha, \qquad f_4 = 3 > \alpha^2.$$

- **归纳步**：假设对所有 $3 \leq j \leq k$，有 $f_j > \alpha^{j-2}$。要证

$$f_{k+1} > \alpha^{k-1}.$$

因为 α 是 $x^2 - x - 1 = 0$ 的正根，所以 $\alpha^2 = \alpha + 1$。于是

$$\alpha^{k-1} = \alpha^2 \alpha^{k-3} = (\alpha+1)\alpha^{k-3} = \alpha^{k-2} + \alpha^{k-3}.$$

当 $k+1 \geq 5$ 时，由归纳假设，

$$f_k > \alpha^{k-2}, \qquad f_{k-1} > \alpha^{k-3}.$$

因此

$$f_{k+1} = f_k + f_{k-1} > \alpha^{k-2} + \alpha^{k-3} = \alpha^{k-1}.$$

3.4 Euclidean Algorithm 与 Lame 定理

若 a, b 是正整数且 $a \geq b$，令

$$a = bq + r, \qquad 0 \leq r < b.$$

则

$$\gcd(a, b) = \gcd(b, r).$$

不断使用带余除法：

$$\begin{aligned}
r_0 &= r_1 q_1 + r_2, && 0 \leq r_2 < r_1, \\
r_1 &= r_2 q_2 + r_3, && 0 \leq r_3 < r_2, \\
&\vdots \\
r_{n-2} &= r_{n-1} q_{n-1} + r_n, && 0 \leq r_n < r_{n-1}, \\
r_{n-1} &= r_n q_n.
\end{aligned}$$

最终

$$\gcd(a,b) = r_n.$$

例：求 $\gcd(414, 662)$

因为 $\gcd(414,662) = \gcd(662,414)$，计算：

$$\begin{aligned} 662 &= 414 \cdot 1 + 248, \\ 414 &= 248 \cdot 1 + 166, \\ 248 &= 166 \cdot 1 + 82, \\ 166 &= 82 \cdot 2 + 2, \\ 82 &= 2 \cdot 41. \end{aligned}$$

因此

$$\gcd(414, 662) = 2.$$

Lame 定理

若 a, b 是正整数且 $a \geq b$，则 Euclidean algorithm 求 $\gcd(a,b)$ 所需除法次数不超过 5 乘以 b 的十进制位数。

证明要点：设算法进行了 n 次除法，$b = r_1$，且 Fibonacci 数列为 $f_0 = 0, f_1 = 1$。由余数递减关系可得

$$r_n \geq 1 = f_2, \quad r_{n-1} \geq 2 = f_3, \quad r_{n-2} \geq f_4, \quad \ldots, \quad b = r_1 \geq f_{n+1}.$$

又由上面的 Fibonacci 下界，

$$f_{n+1} > \alpha^{n-1},$$

所以

$$b > \alpha^{n-1}.$$

取常用对数，并利用 $\log_{10} \alpha > 1/5$，得

$$\log_{10} b > (n-1)\log_{10} \alpha > \frac{n-1}{5}.$$

若 b 有 k 位十进制数字，则 $\log_{10} b < k$，从而

$$n - 1 < 5 \log_{10} b < 5k.$$

因为 n 为整数，所以 $n \leq 5k$。

3.5 递归定义集合与结构

集合也可以递归定义：

1. **基础步**：指定最初属于集合的元素；
2. **递归步**：给出由已有元素构造新元素的规则。

例：整数子集

定义集合 S：

$$3 \in S, \qquad x \in S \wedge y \in S \Rightarrow x + y \in S.$$

直观上，S 是所有正的 3 的倍数。

例：字符串集合 Σ^*

设 Σ 是字母表，Σ^* 表示由 Σ 中符号构成的所有有限字符串。

$$\lambda \in \Sigma^*, \qquad w \in \Sigma^*,\ x \in \Sigma \Rightarrow wx \in \Sigma^*.$$

其中 λ 表示空串。

例：字符串连接与长度

连接运算 $\bullet$ 可递归定义为

$$w \bullet \lambda = w,$$

$$w_1 \bullet (w_2 x) = (w_1 \bullet w_2)x,$$

其中 $w, w_1, w_2 \in \Sigma^*,\ x \in \Sigma$。

字符串长度函数 $\ell(w)$ 的递归定义为

$$\ell(\lambda) = 0, \qquad \ell(wx) = \ell(w) + 1.$$

3.6 合式公式的递归定义

命题逻辑合式公式

含有 T, F、命题变量以及联结词 $\{\leftrightarrow, \rightarrow, \neg, \wedge, \vee\}$ 的合式公式可递归定义：

- **基础步**：T, F 与任意命题变量 p 都是合式公式。
- **递归步**：若 p, q 是合式公式，则

  $$(\neg p), \quad (p \vee q), \quad (p \wedge q), \quad (p \rightarrow q), \quad (p \leftrightarrow q)$$

  都是合式公式。

表达式合式公式

由变量、数值以及运算符 $\{+, -, *, /, \uparrow\}$ 构成的表达式可递归定义：

- **基础步**：任意变量或数值 x 是合式公式。
- **递归步**：若 F, G 是合式公式，则

  $$(F + G), \quad (F - G), \quad (F * G), \quad (F/G), \quad (F \uparrow G)$$

  都是合式公式。

3.7 递归定义树

有根树 (Rooted Trees)

有根树由一个特殊顶点 root 和连接顶点的边构成。递归定义为：

- **基础步**：单个顶点 r 是一棵有根树。
- **递归步**：若 $T_1, T_2, \ldots, T_n$ 是两两不交的有根树，其根分别为 $r_1, r_2, \ldots, r_n$，取一个不属于这些树的新顶点 r，并添加边

$$(r, r_1), (r, r_2), \ldots, (r, r_n),$$

得到的新图仍是有根树。

满二叉树 (Full Binary Trees)

满二叉树可递归定义为：

- **基础步**：只有一个根 r 的树是一棵满二叉树。
- **递归步**：若 T_1, T_2 是两棵不交的满二叉树，则 $T_1 \bullet T_2$ 表示由一个新根 r 连接到 T_1 与 T_2 的根后得到的满二叉树，其中 T_1 是左子树，T_2 是右子树。

3.8 结构归纳法

对递归定义的集合或结构证明性质时，通常使用**结构归纳法 (structural induction)**。

结构归纳法模板

1. **基础步**：证明性质对递归定义基础步中直接给出的所有元素成立。
2. **递归步**：假设性质对构造新元素所用的已有元素成立，证明性质对新构造出的元素也成立。

$$\text{数学归纳原理} + \text{递归定义} \Longrightarrow \text{结构归纳法}.$$

3.9 例 9：递归集合 S 是正的 3 的倍数集合

令

$$A = \{n \in \mathbb{Z}^+ \mid 3 \mid n\}.$$

要证明 $S = A$，需证 $S \subseteq A$ 和 $A \subseteq S$。

- $S \subseteq A$：用结构归纳法。基础元素 3 可被 3 整除。若 x, y 都可被 3 整除，则 $x + y$ 也可被 3 整除。因此 S 中每个元素都是正的 3 的倍数。
- $A \subseteq S$：用普通归纳法证明对每个正整数 m，$3m \in S$。基础步 $3 \in S$；若 $3m \in S$，则 $3(m+1) = 3m + 3 \in S$。

3.10 例 10：合式公式中的括号数量

证明：每个命题逻辑合式公式中，左括号数等于右括号数。

证明：用结构归纳法。

- **基础步**：T, F 和命题变量 p 中左右括号数都为 0。
- **递归步**：若性质对合式公式 p, q 成立，则

$$(\neg p), \quad (p \vee q), \quad (p \wedge q), \quad (p \to q), \quad (p \leftrightarrow q)$$

都是在已有公式外增加一对左右括号，或组合两个已有公式后再增加一对左右括号。因此左右括号数仍相等。

3.11 例 11：字符串长度的连接公式

证明对任意 $x, y \in \Sigma^*$，

$$\ell(xy) = \ell(x) + \ell(y).$$

证明：固定 x，对字符串 y 的递归定义做结构归纳。令 $P(y)$ 表示上述等式成立。

- **基础步**：

$$\ell(x\lambda) = \ell(x) = \ell(x) + 0 = \ell(x) + \ell(\lambda).$$

- **递归步**：假设 $P(y)$ 成立。对任意 $a \in \Sigma$，

$$\ell(xya) = \ell(xy) + 1 = \ell(x) + \ell(y) + 1 = \ell(x) + \ell(ya).$$

3.12 例 12：满二叉树的顶点数上界

满二叉树 T 的高度 $h(T)$ 递归定义为：

$$h(T) = 0 \quad \text{若} T \text{ 只有一个根},$$

$$h(T_1 \bullet T_2) = 1 + \max(h(T_1), h(T_2)).$$

顶点数 $n(T)$ 递归定义为：

$$n(T) = 1 \quad \text{若} T \text{ 只有一个根},$$

$$n(T_1 \bullet T_2) = 1 + n(T_1) + n(T_2).$$

证明：

$$n(T) \le 2^{h(T)+1} - 1.$$

证明：使用结构归纳法。

- **基础步**：若 T 只有一个根，则 $n(T) = 1$，$h(T) = 0$，故

$$n(T) = 1 \le 2^{0+1} - 1 = 1.$$

- **递归步**：设 $T = T_1 \bullet T_2$，并假设

$$n(T_1) \le 2^{h(T_1)+1} - 1, \qquad n(T_2) \le 2^{h(T_2)+1} - 1.$$

则

$$\begin{aligned} n(T) &= 1 + n(T_1) + n(T_2) \\ &\le 1 + \left(2^{h(T_1)+1} - 1\right) + \left(2^{h(T_2)+1} - 1\right) \\ &\le 2\max\left(2^{h(T_1)+1}, 2^{h(T_2)+1}\right) - 1 \\ &= 2 \cdot 2^{\max(h(T_1),h(T_2))+1} - 1 \\ &= 2^{h(T)+1} - 1. \end{aligned}$$

3.13 广义归纳法 (Generalized Induction)

归纳思想可推广到整数以外的良序集合。若一个集合上的次序保证每个非空子集都有最小元素，就可以对该次序使用归纳证明。

字典序

在 $\mathbb{N} \times \mathbb{N}$ 上可定义字典序：

$$(x_1, y_1) \le (x_2, y_2)$$

当且仅当 $x_1 < x_2$，或 $x_1 = x_2$ 且 $y_1 \le y_2$。该思想可推广到 k 维元组。

例：二元递归序列

对 $(m, n) \in \mathbb{N} \times \mathbb{N}$，递归定义

$$a_{0,0} = 0,$$

$$a_{m,n} = \begin{cases} a_{m-1,n} + 1, & n = 0,\ m > 0, \\ a_{m,n-1} + n, & n > 0. \end{cases}$$

证明

$$a_{m,n} = m + \frac{n(n+1)}{2}.$$

证明：按 $\mathbb{N} \times \mathbb{N}$ 的字典序归纳。

- **基础步**：$(m, n) = (0, 0)$ 时，

$$a_{0,0} = 0 = 0 + \frac{0(0+1)}{2}.$$

- **递归步**：假设所有字典序小于 (m, n) 的位置都满足公式。
  - 若 $n = 0$ 且 $m > 0$，则

$$a_{m,0} = a_{m-1,0} + 1 = (m-1) + 1 = m.$$

  - 若 $n > 0$，则

$$a_{m,n} = a_{m,n-1} + n = m + \frac{(n-1)n}{2} + n = m + \frac{n(n+1)}{2}.$$

课件练习

Page 378: 24, 30, 37, 38, 46, 47。

3.14 递归算法 (Recursive Algorithms)

3.15 定义与设计方法

递归算法通过把原问题化为同类但输入规模更小的问题来求解。
设计递归算法时，通常需要明确：

- **基本情形 (base case)**：小到可以直接求解的输入；
- **递归调用 (recursive call)**：把问题缩小；
- **合并方式**：如何用较小问题的答案得到原问题的答案；
- **终止性**：每次递归都必须朝基本情形靠近。

3.16 例 13：递归计算幂

设 a 是非零实数，n 是非负整数。

```
procedure power(a, n)
    if n = 0 then
        return 1
    else
        return a * power(a, n - 1)
```

3.17 例 14：递归计算模幂

设 $m \geq 2$，$n \geq 0$，$1 \leq b \leq m$。利用

$$(xy) \bmod m = \big((x \bmod m)(y \bmod m)\big) \bmod m$$

以及快速幂思想：

$$b^n \bmod m = \begin{cases} 1, & n = 0, \\ \left(b^{n/2} \bmod m\right)^2 \bmod m, & n > 0 \text{ 且} n \text{ 为偶数}, \\ \left(\left(b^{\lfloor n/2 \rfloor} \bmod m\right)^2 \bmod m \cdot b\right) \bmod m, & n \text{ 为奇数}. \end{cases}$$

```
procedure mpower(b, n, m)
    if n = 0 then
        return 1
    else if n is even then
        t := mpower(b, n / 2, m)
```

```
        return (t * t) mod m
    else
        t := mpower(b, floor(n / 2), m)
        return ((t * t mod m) * b) mod m
```

3.18 例 15：递归计算最大公约数

对非负整数 a, b，且 $a < b$:

```
procedure gcd(a, b)
    if a = 0 then
        return b
    else
        return gcd(b mod a, a)
```

其正确性来自

$$\gcd(a, b) = \gcd(b \bmod a, a).$$

3.19 例 16：递归二分查找

设有递增序列 $a_1, a_2, \ldots, a_n$，目标为 x。

```
procedure binary_search(x, i, j)
    m := floor((i + j) / 2)
    if x = a[m] then
        location := m
    else if x < a[m] and i < m then
        binary_search(x, i, m - 1)
    else if x > a[m] and j > m then
        binary_search(x, m + 1, j)
    else
        location := 0
```

3.20 递归与迭代

同一问题常有递归和迭代两种写法。

阶乘

递归版本

```
procedure factorial(n)
    if n = 1 then
        return 1
    else
        return n * factorial(n - 1)
```

迭代版本

```
procedure iterative_factorial(n)
    x := 1
    for i := 1 to n
        x := i * x
    return x
```

Fibonacci 数

直接递归版本：

```
procedure fibonacci(n)
    if n = 0 then
        return 0
    else if n = 1 then
        return 1
    else
        return fibonacci(n - 1) + fibonacci(n - 2)
```

该算法会重复计算大量相同子问题，效率较低。标准迭代版本只需线性时间：

```
procedure iterative_fibonacci(n)
    if n = 0 then
        return 0
    x := 0
    y := 1
    for i := 2 to n
        z := x + y
        x := y
        y := z
    return y
```

3.21 归并排序 (Merge Sort)

归并排序的思想：

- 将列表反复分成两个子列表，直到每个子列表只有一个元素；
- 每个子列表可视为一棵平衡二叉树中的节点；

- 自底向上不断合并两个已经有序的子列表；
- 最终得到整体有序列表。

例：排序列表 $8, 2, 4, 6, 9, 7, 10, 1, 5, 3$

阶段	列表状态
原始列表	$[8, 2, 4, 6, 9, 7, 10, 1, 5, 3]$
拆分	$[8, 2, 4, 6, 9]$ $[7, 10, 1, 5, 3]$
继续拆分	$[8, 2, 4]$ $[6, 9]$ $[7, 10, 1]$ $[5, 3]$
单元素附近	$[8]$ $[2]$ $[4]$ $[6]$ $[9]$ $[7]$ $[10]$ $[1]$ $[5]$ $[3]$
合并	$[2, 8]$ $[4]$ $[6, 9]$ $[7, 10]$ $[1]$ $[3, 5]$
继续合并	$[2, 4, 8]$ $[6, 9]$ $[1, 7, 10]$ $[3, 5]$
继续合并	$[2, 4, 6, 8, 9]$ $[1, 3, 5, 7, 10]$
最终结果	$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

递归归并排序算法

```
procedure mergesort(L = a[1], ..., a[n])
    if n > 1 then
        m := floor(n / 2)
        L1 := a[1], ..., a[m]
        L2 := a[m + 1], ..., a[n]
        L := merge(mergesort(L1), mergesort(L2))
    return L
```

合并两个有序列表

```
procedure merge(L1, L2)
    L := empty list
    while L1 and L2 are both nonempty
        remove the smaller first element from L1 or L2
        append it to the right end of L
    append all remaining elements of the nonempty list to L
    return L
```

复杂度定理：归并排序一个含 n 个元素的列表所需比较次数为

$$O(n \log n).$$

理由是：每一层合并总工作量为 $O(n)$，平衡拆分产生 $O(\log n)$ 层。

课件练习

Page 391: 12, 45, 50。

4 程序正确性 (Program Correctness)

课件指出：为了确信程序总能给出正确输出，需要证明程序正确性；但对复杂程序进行完全机械化证明仍然困难。离散数学中通常先学习小程序和算法的正确性证明方法。

4.1 基本概念

- **前置条件 (precondition)**：程序开始前必须满足的条件。
- **后置条件 (postcondition)**：程序正常结束后应满足的条件。
- **部分正确性 (partial correctness)**：若程序在满足前置条件时终止，则终止后的结果满足后置条件。
- **完全正确性 (total correctness)**：程序部分正确，并且一定会终止。

常用 Hoare 三元组表示为

$$\{P\}\ S\ \{Q\},$$

含义是：若执行语句或程序段 S 前 P 成立，且 S 终止，则执行后 Q 成立。

4.2 循环不变式

证明循环正确性常用**循环不变式 (loop invariant)**。

1. **初始化**：循环第一次执行前，不变式成立。
2. **保持**：若某次循环前不变式成立，则执行循环体后仍成立。
3. **终止**：循环结束时，不变式与循环终止条件一起推出后置条件。

例：迭代阶乘的正确性

程序：

```
x := 1
for i := 1 to n
    x := i * x
return x
```

循环不变式可取为：完成第 i 次循环后，

$$x = i!.$$

- 初始化：循环前 $x = 1 = 0!$。

- 保持：若第 $i-1$ 次后 $x=(i-1)!$，第 i 次执行 $x := i \cdot x$ 后有 $x = i!$。
- 终止：当 $i = n$ 的循环完成后，$x = n!$，因此返回值正确。

4.3 递归程序正确性

递归算法的正确性通常通过归纳证明：

- **基础情形**：证明 base case 直接返回正确答案。
- **归纳假设**：假设递归调用在所有更小输入上正确。
- **归纳步**：证明当前输入通过递归调用和合并步骤得到正确答案。
- **终止性**：证明每次递归调用的输入规模严格减小，最终到达 base case。

例如，Euclidean algorithm 的终止性来自余数严格变小；归并排序的终止性来自子列表长度严格小于原列表。

5 全章总结

1. **普通归纳法**适合“由 n 推 $n+1$”的命题。
2. **强归纳法**适合当前对象依赖多个更小对象的证明，如素数分解、邮票问题和递归序列估计。
3. **良序性**是归纳法的根基，也支撑带余除法、最短反例法和无限递降法。
4. **递归定义**由基础步和递归步组成，可定义函数、集合、字符串、公式、树等对象。
5. **结构归纳法**是递归定义对象上的自然证明方法。
6. **递归算法**必须同时关注正确性与终止性；很多递归算法也可改写为迭代形式。
7. **程序正确性**通过前置条件、后置条件、循环不变式和归纳证明来刻画。

章节	主题	课件练习
5.1	Mathematical Induction	P350: 44, 61
5.2	Strong Induction and Well-Ordering	P363: 12, 25, 31, 37
5.3	Recursive Definitions and Structural Induction	Page 378: 24, 30, 37, 38, 46, 47
5.4	Recursive Algorithms	Page 391: 12, 45, 50
5.5	Program Correctness	结合循环不变式与递归正确性复习

归纳法证明“所有有限步都能到达”，递归法说明“每一步如何构造”。